ImageLab Assessment 2

Short- and long-polling comparison
Daryn Forman | Report: 26 October 2026 | Experiments: 25 October 2026

1 Predictions recorded before testing

A — Quiet job: With a 30-second delay before real transformations, I expected short polling to send more requests because long polling avoids unnecessary checking.

B — Real progress: With real transformations only, I expected short polling to use fewer requests because the client stops when it detects completion.

C1 — Slow updates: With a 10-second simulated phase updating every 2 seconds, I expected long polling to use fewer requests because it waits for updates.

C2 — Fast updates: With a 10-second simulated phase updating every 200 ms, I expected short polling to use fewer requests because each check could reflect several intervening updates.

C3 — Fast updates with latency: Repeating C2 with added latency, I expected both modes' detection delays to increase because responses take longer to arrive, with roughly the same request counts as C2.

2 Experimental method

Thirty runs used the real Firefox UI, Go API, PostgreSQL and one worker: five conditions × two modes × three repetitions. The recorder automated uploads and read the existing client log. Configurations recorded the same VALID-beach.png hash and transformation settings: 150 × 150 crop, preview within 800 × 600, and display within 1200 × 900. Final metadata recorded all three variants in every run.

The recorder checked for zero queued/processing jobs before each upload and used one controlled observer. Separate SQL check output was not retained. Each condition alternated mode-first order: short/long, long/short, short/long. Visibility samples stayed foreground; continuous visibility was not recorded.

Both modes began with a prompt snapshot, whose version became the baseline. Short polling waited 1,000 ms after each response/render, rather than between request starts; logs show no overlap. Long polling immediately renewed active responses, holding up to 20 seconds, with a 30-second client timeout. The window ran from first observation GET start to terminal JSON receipt, excluding upload and timing probes.

Detection delay = browser Date.now() at terminal JSON receipt − server wall-clock log immediately after terminal commit, matched by job ID and version on the same computer. This estimates availability-to-receipt delay; rendering timestamps record DOM updates, not screen paint. Processing duration = completed_at − started_at, including configured waits: 30 seconds in A and the 10-second simulated phase in C1–C3. Actual commits were logged; the UI labelled simulated progress; simulation_step stayed separate from real variants_completed and preceded transformations.

Only C3 used Firefox Network Monitor throttling: latency 200 ms, upload/download limits 1,000,000,000 bytes/s. Before each upload, five prompt health-GET timings with and without throttling gave median increases of 174–285 ms (baseline 6–51 ms; throttled 216–292 ms). This was approximate added latency, not exactly 200 ms. Other conditions had no configured added latency.

3 Measured results

Table 1. Median (minimum–maximum), three runs per cell. GETs count request starts, including the initial snapshot; uploads and health probes are excluded. Delay is measured in milliseconds (ms).

Condition	GETs Short	GETs Long	Delay (ms) Short	Delay (ms) Long
A	32 (31–32)	6 (6–6)	693 (128–773)	13 (3–21)
B	2 (2–2)	5 (5–6)	111 (29–221)	10 (7–12)
C1	12 (12–12)	10 (10–11)	427 (102–905)	8 (7–39)
C2	12 (12–12)	55 (55–56)	346 (323–830)	12 (4–14)
C3	10 (9–10)	44 (42–45)	542 (426–1221)	425 (415–484)

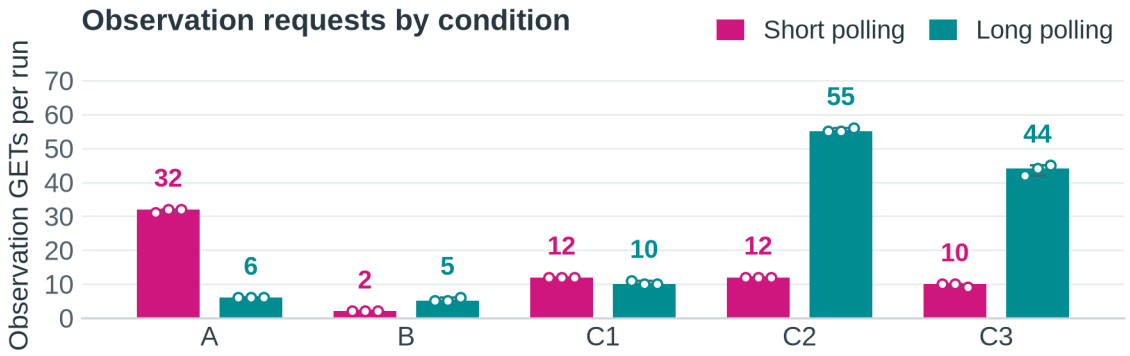


Figure 1. Bars and numeric labels show median GETs per run; dots show the three runs and lines show minimum–maximum. A: quiet; B: real; C1: slow updates; C2: fast updates; C3: fast updates with latency.

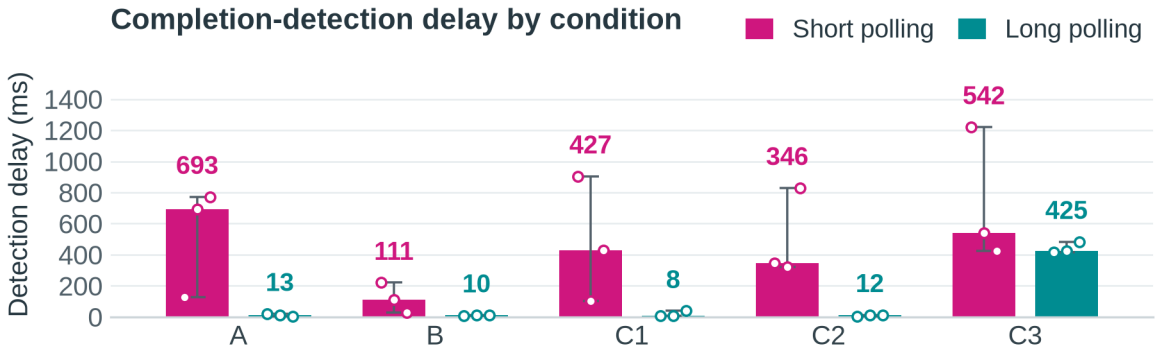


Figure 2. Bars and numeric labels show median detection delay (ms); dots show individual runs and lines show minimum–maximum. C3 prompt-GET median increases were 174–285 ms. Conditions match Figure 1.

Version coverage — median (minimum–maximum) distinct versions received after baseline: C2 short 11 (11–11), long 54 (54–55); C3 short 9 (8–9), long 43 (41–44). Missing committed versions: C2 short 44 (44–44), long 1 (0–1); C3 short 46 (45–46), long 12 (11–14). C2 baselines were 1; C3 short baselines were 2, 1, 2 and long baselines were 1. All terminal versions were 56. All 30 runs recorded completion, three variants and zero observation errors, cancellations or pending requests. Full metrics remain in runs.csv; the incomplete setup attempt is excluded.

4 Interpretation and limitations

Long polling helped during quiet work: A fell from 32 to 6 median GETs and from 693 to 13 ms median detection delay. Across A short runs, 86 of 95 GETs were unchanged. C1 also used fewer long-poll GETs (10 versus 12), supporting my A and C1 request predictions.

During ordinary processing in B, median completion-detection delay was 111 ms for short polling and 10 ms for long polling. Short polling used two GETs, while long polling observed more milestones. Both stop at completion, so my B explanation was not unique to short polling. C2 supported my request prediction (12 versus 55 median GETs), but responses are current snapshots, not event lists. C3 raised both median delays relative to C2; counts fell to 10 and 44, contrary to my roughly unchanged forecast. Longer response cycles span more commits, consistent with the version gaps on page 2. Long polling had lower median delay throughout, but C3 repetition 3 favoured short polling: 426 versus 484 ms.

These results concern observation, not faster image processing: the worker performs the same transformations independently. B processing-duration medians were 217.594 ms (short) and 451.356 ms (long), with only three runs each. Subtracting configured waits in A and C1–C3 would still leave database and scheduling time, not pure transformation time.

Fewer HTTP requests need not mean fewer SQL queries: C1 median observation reads were 13 (short) and 20 (long), despite fewer long-poll GETs. This handler waits on notifications, not periodic SQL polling; reads occur on entry, wake-up and deadline recheck, plus completed-result metadata reads. Counters exclude worker queries; held-handler memory, CPU and connection reuse were not measured.

Synchronization order: register each observer's buffered signal under a mutex, then read PostgreSQL. Return if newer or terminal; otherwise wait. A commit before the read is visible there; one between reading and waiting leaves a buffered signal. The worker persists state/progress and the increased version atomically, then broadcasts only after commit. Every awakened observer rereads PostgreSQL; the signal does not prove success. The deadline also triggers a final read before 204. This order prevents the check-to-wait gap from hiding a change.

In A-long-1, a 204 followed a 20.044-second request; the next GET began in the same receipt millisecond with after=2 unchanged. After active 200, render, save the version and immediately request again; after 204, retain the version and renew; terminal responses stop observation. Versions 1, 2, 3, 4 and 6 arrived: missing 5 did not lose the final result. If every event mattered, durable event history and replay would be needed. A continuing response could send separately framed updates for incremental browser reading, avoiding a new GET per update; this was not implemented.

Limitations: three repetitions, one image and one computer; millisecond timestamps, JSON parsing, logging after commit and scheduling uncertainty. C3 alone enabled Firefox monitoring, a possible tooling confound, and latency was approximate. Foreground sampling and unsaved queue-check output limit independent verification. Successful runs do not establish all failure/lifecycle checks, scaling performance or statistical significance.

5 Recommendation

Use long polling when prompt feedback matters and changes are occasional, as in A and C1. Keep short polling available when roughly one-second checks are acceptable and frequent updates would create many long-poll requests. The choice should follow feedback needs and measured server cost; neither mode is universally more efficient.

Evidence: runs.csv and matching *-client.json, *-server.log, *-snapshot.json, *-summary.json and *-prompt-get.json files in the accompanying evidence archive. Scope and definitions follow the teacher's Assessment 2 brief and ImageLab Version 2 Requirements.